

Iniziato	martedì, 13 febbraio 2024, 15:36
Stato	Completato
Terminato	martedì, 13 febbraio 2024, 17:02
Tempo impiegato	1 ora 26 min.
Valutazione	15,17 su un massimo di 20,00 (75,83%)
Feedback	Thanks for your attempt, all the best!

Domanda 1

Parzialmente corretta

Punteggio ottenuto 0,75 su 1,00

For the following tasks specify if you can train the model in an unsupervised fashion or you necessarily need to do it in a supervised fashion, i.e., asking something to an expert labeler.

Supervised	Unsupervised		
☒	☐	A deep neural network for autonomous driving which recognizes where in the image are located pedestrians and cars.	✓ You need the bounding boxes or the masks otherwise you cannot train the model
☐	☒	A deep neural network for time series prediction which is able to predict the next "acqua alta" (namely "high water") event in Venice. Assume you have just a time series of events in the last 20 years.	✓ You just need the time series of the "acqua alta" phenomenon and no supervision is required
☒	☐	A deep neural model which is able, from an image of a cow, to tell if the cow is from the Frisone type or not.	✗ It is an image classification task for which you need a labeled dataset with cows of different kinds including the Frisone one
☐	☒	A deep neural model to perform image restoration / inpainting, i.e., given an image with some obscured pieces can restore the values of the missing pixels.	✓ You can train this taking images with no issues, remove random pieces and restore them with an autoencoder architecture.

A deep neural network for autonomous driving which recognizes where in the image are located pedestrians and cars. : Supervised

A deep neural network for time series prediction which is able to predict the next "acqua alta" (namely "high water") event in Venice. Assume you have just a time series of events in the last 20 years. : Unsupervised

A deep neural model which is able, from an image of a cow, to tell if the cow is from the Frisone type or not.: Supervised

A deep neural model to perform image restoration / inpainting, i.e., given an image with some obscured pieces can restore the values of the missing pixels. : Unsupervised

Domanda **2**

Completo

Punteggio ottenuto 4,00 su 4,00

The Vanishing Gradient is a well-known issue in deep neural networks. Answer the following questions in a focused and concise manner!

a) What is the vanishing gradient issue?

b) What is vanishing gradient due to?

c) Which network architectures are particularly affected by vanishing gradient

d) Which techniques can be used to limit the vanishing gradient issue

a) the vanishing gradient is an issue present in many types of neural networks that is due to their intrinsic architecture. Vanishing gradient could cause the network not to learn anything since weights get to (or very close to) 0.

b) During training the network uses a gradient descent algorithm (i.e. SGD or Adam) which compute the updates of the weights based on the gradient of the error function on the input variable. Using the most common activation functions (tanh, sigmoid, relu) effectively multiplies the values by a value which is could get very close to 0 and this could cascade all through the network making it impossible to learn anything.

c) The more a network is deep the more is affected by the vanishing gradient since at every layer we have the mutiplications, which make the gradient get closer and closer to zero. Classic RNN networks are the most sensitive as they could be very deep since we have to "unroll" them for every timestep we want to memorize.

d) In order to solve the VG problem in RNN the best thing we can do is to drop the classic RNN networks and switch to for example LSTM cells, which are created to tackle this exact issue using the Constant-Error-Carusel which keep the gradient to 1 during the unroll. In case of non RNN we cause use for example the BatchNormalization to normalize the weights in a layer in order to try to mitigate the problem, we can also change the weight init function or choose other activation functions.

Commento:

Domanda 3

Risposta corretta

Punteggio ottenuto 3,00 su 3,00

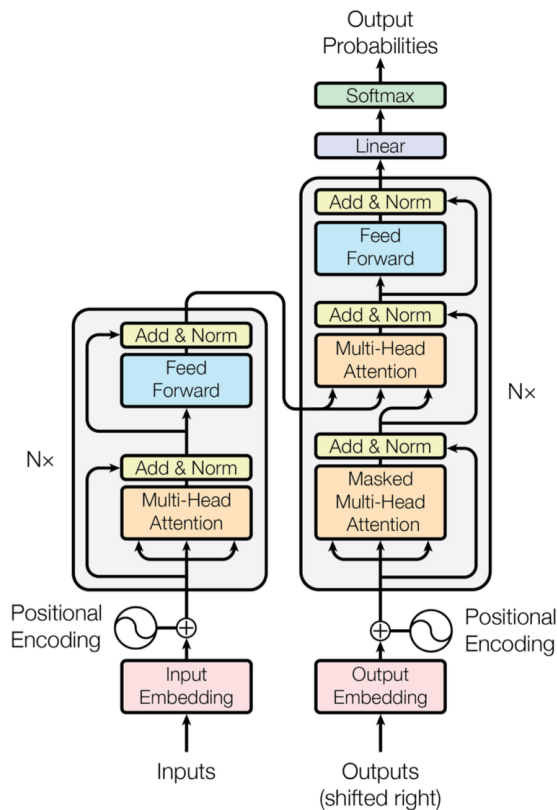


Figure 1: The Transformer - model architecture.

Let's consider the Transformer model in the picture and the blocks composing it. For each of the following statements, identify if the statement is true or false.

Please notice that the answer has to be based on technical considerations, not on the particular way the question is formulated.

True **False**

☒ ☐

☐ ☒

Provided the same input sequence twice, in two different executions one after the other, a transformer generates twice the same output sequence.

✓

Yes, a transformer like most of the deep models is a deterministic one and thus given the same input twice it produces twice the same deterministic output.

☒ ☐

☐ ☒

In the Multi-Head Attention, it is possible to "cross" Query/Key/Values coming from the encoder and the decoder

✓

Yes, this is the mechanism used to condition the decoder on the outcome of the encoder.

True	False		
☐	☒	The self-attention layer is said to be "permutation invariant". Does this mean that when we provide as input to the self-attention layer a sentence reversed, we obtain the same result of the sentence in the forward direction?	✓ No, we get the output reversed, permutation invariant means self-attention does not take into account the position of the token
☐	☒	Transformers use a position encoding mechanism that is based on sinusoidal and cosinusoidal functions of the token positions added to the encoding of the input tokens. Although this might seem counterintuitive it is successfully used by all transformers when they use positional encoding.	✓ No, some transformer implementations use learned positional encoders and they can also use concatenation instead of sum with the input tokens.
☐	☒	Transformers require to have both encoder and decoder stages, i.e., there cannot be "encoder only" or "decoder only" Transformer models, otherwise it might not be possible to train them.	✓ False, BERT is a pure encoder base Transformer while GPT is a pure decoder based Transformer
☒	☐	We can have very deep Transformer architectures and still be able to train those. Indeed, residual connections which compose all Transformer building blocks limit the vanishing gradient issue.	✓

Provided the same input sequence twice, in two different executions one after the other, a transformer generates twice the same output sequence.: True

In the Multi-Head Attention, it is possible to "cross" Query/Key/Values coming from the encoder and the decoder : True

The self-attention layer is said to be "permutation invariant". Does this mean that when we provide as input to the self-attention layer a sentence reversed, we obtain the same result of the sentence in the forward direction?

: False

Transformers use a position encoding mechanism that is based on sinusoidal and cosinusoidal functions of the token positions added to the encoding of the input tokens. Although this might seem counterintuitive it is successfully used by all transformers when they use positional encoding.: False

Transformers require to have both encoder and decoder stages, i.e., there cannot be "encoder only" or "decoder only" Transformer models, otherwise it might not be possible to train them.: False

We can have very deep Transformer architectures and still be able to train those. Indeed, residual connections which compose all Transformer building blocks limit the vanishing gradient issue.: True

Domanda 4

Parzialmente corretta

Punteggio ottenuto 0,33 su 2,00



Clever Hans (German: der Kluge Hans; c. 1895 – c. 1916) was a horse that was claimed to have performed arithmetic and other intellectual tasks. After a formal investigation in 1907, psychologist Oskar Pfungst demonstrated that the horse was not performing these mental tasks, but was *watching the reactions of his trainer Wilhelm von Osten*.

In their spectacles, Von Osten would ask Hans, "If the eighth day of the month comes on a Tuesday, what is the date of the following Friday?" Hans would answer by tapping his hoof eleven times in a row. Indeed, as the horse's tap sequence approached the right answer, i.e., the right number of taps, the questioner's posture and facial expression changed in ways that were consistent with an increase in tension, which was released when the horse made the final, correct tap providing a cue to tell it to stop the sequence of taps.

Let's assume you want to train a deep neural model to implement the tapping behavior of Clever Hans, i.e., producing a sequence of taps until "something" happens in the posture/face of a person a camera is capturing. Which of the following are proper statements about architectural choices for a neural network model to implement this behavior?

NOTE: Random guessing leads to **score 0 on average**. If you get all the correct answers you get the full mark, if you get all wrong answers you can get a negative score. A wrong answer is a statement you should have checked and you did not, or a statement you should not have checked and you did.

Each statement has to be considered independently not taking into account any of the others, and we are not interested in things technically possible in python/tensorflow/keras, we want you to show your understanding of classical design choices that have been described in the course.

Scegli una o più alternative:

- ☐ a. The model is trained by giving pairs of sequences, i.e., the question the horse needs to answer and the sequence of taps to produce. The loss function is the cross-entropy, indeed this is a classical sequence-to-sequence model.
- ☒ b. The desired behavior could be implemented with a CNN observing the face of the trainee when asking the question and directly regressing the number of taps to be done. ✗ No, it is a sequential problem, not one shot, it generates a sequence of taps.

- ☒ c. The model could be implemented with a recurrent neural network having images as input, possibly pre-processed by some convolutional layers, and a sigmoidal output with 2 possible output, i.e., symbols {tap, <eos>}. ✔ Yes, this is exactly the model described by the text
- ☒ d. The model can be trained to decide for the current output with binary cross-entropy as the output is basically "tap" or "don't tap". ✔ Yes, indeed the horse has to tap until detects the change in the posture/face of the trainee and then stops.
- ☐ e. The model needs a sequence of images as input, thus it is likely to have some convolutional layers composing it.

Your answer is partially correct.

Hai selezionato correttamente 2.

Le risposte corrette sono:

The model could be implemented with a recurrent neural network having images as input, possibly pre-processed by some convolutional layers, and a sigmoidal output with 2 possible output, i.e., symbols {tap, <eos>}. ,

The model can be trained to decide for the current output with binary cross-entropy as the output is basically "tap" or "don't tap".,

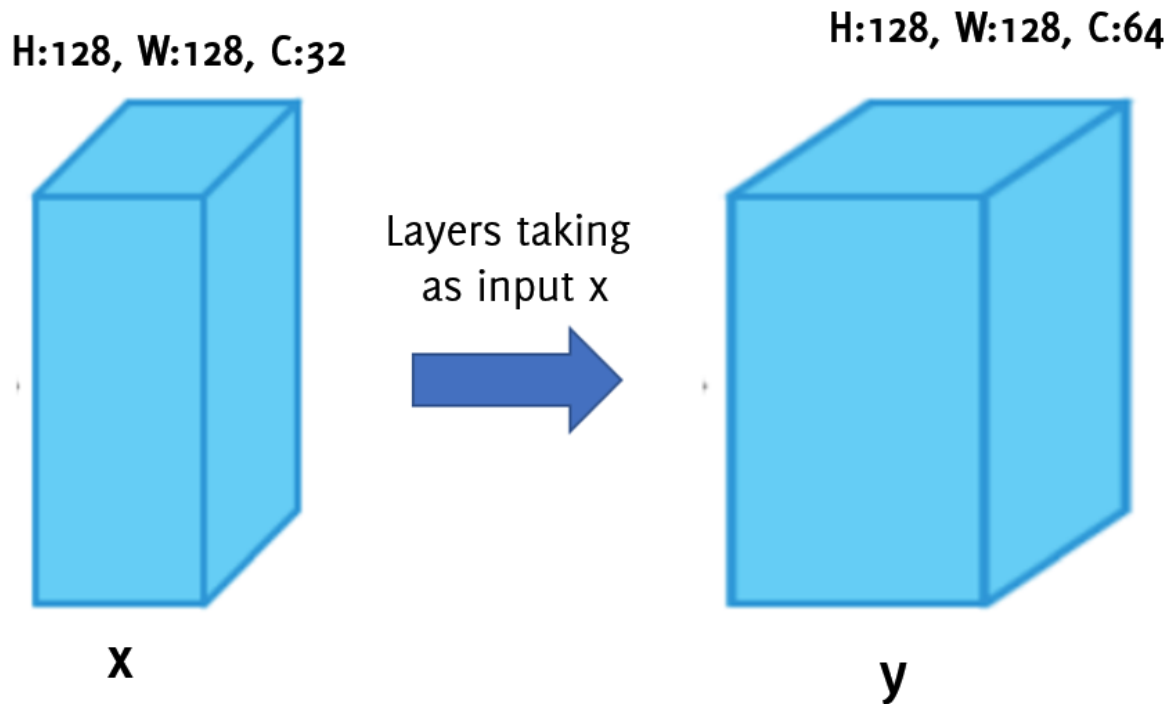
The model needs a sequence of images as input, thus it is likely to have some convolutional layers composing it.

Domanda 5

Parzialmente corretta

Punteggio ottenuto 0,67 su 1,00

Which of the following layers can be technically plugged in between the activations maps in the following scheme



Scegli una o più alternative:

- ☒ a. `y = tfkl.Conv2D(64, kernel_size=1, padding='valid', activation='relu')(x)` ✓ TRUE!
- ☒ b. `x = tfkl.Conv2D(128, kernel_size=3, padding='same')(x)` ✓ TRUE
`y = tfkl.Conv2D(64, kernel_size=3, padding='same')(x)`
- ☐ c. `y = tfkl.Activation(activations.relu)(x)`
- ☐ d. `x = tfkl.Conv2D(64, kernel_size=3, padding='same', activation='relu')(x)`
`y = tfkl.MaxPooling2D(x, pool_size=(3, 3), strides = (1,1), padding = 'same')(x)`
- ☐ e. `y = tfkl.Conv2D(32, kernel_size=3, padding='same', activation='relu')(x)`
- ☐ f. `y = tfkl.UpSampling2D()(x)`
- ☐ g. `x1 = tfkl.Conv2D(32, kernel_size=3, padding='same')(x)`
`x2 = tfkl.Conv2D(64, kernel_size=3, padding='same')(x1)`
`y = tfkl.Add([x1,x2])`

- ☐ h. `y = GlobalAveragingPooling2D()(x)`
- ☐ i. it is not possible to say, since we don't know what will happen in the next layers after y

Your answer is partially correct.

Hai selezionato correttamente 2.

Le risposte corrette sono:

```
x = tfkl.Conv2D(64, kernel_size=3, padding='same', activation='relu')(x)
y = tfkl.MaxPooling2D(x, pool_size=(3, 3), strides = (1,1), padding = 'same')(x)
',
y = tfkl.Conv2D(64, kernel_size=1, padding='valid', activation='relu')(x),
x = tfkl.Conv2D(128, kernel_size=3, padding='same')(x)

y = tfkl.Conv2D(64, kernel_size=3, padding='same')(x)
```

Domanda 6

Parzialmente corretta

Punteggio ottenuto 0,75 su 1,00

Check all the following statements which are true. Any wrong answer results in a penalty.

Scegli una o più alternative:

- ☐ a. Augmentation layers are very useful during network training, but they need to be removed during inference, since it is not possible to augment a test image.
- ☐ b. Mixup augmentation consists in computing a convex combination on pairs of input images and feeding this to the network. The target associated to the resulting sample is the label of the image that contributes the most to the convex combination.
- ☒ c. VGG is a very popular CNN architecture for image classification, trained on ImageNet classes. Let's assume you need to train a classifier to perform a specific task (e.g. mammals classification) and you want to take advantage of this pre-trained model. Transfer learning in this case consists in running a few training epochs on the new training set, keeping as trainable only the MLP layers of the VGG trained on Imagenet. ✗ FALSE: you need to get rid of the MLP which is task specific on ImageNet, and replae it with the right output layer for the mammals classification task
- ☐ d. It is possible to plug in any form of photometric or geometric transformation during training, this will be anyhow beneficial
- ☐ e. It is good to perform fine tuning before transfer learning
- ☒ f. In order to perform backpropagation of the maxpooling it is necessary to keep track of the coordinates of all the locations that have propagated the maximum ✓ TRUE; the derivative is a dirac centered there
- ☒ g. It is good to perform transfer learning before fine tuning, as long as there are enough training data. ✓ TRUE

Your answer is partially correct.

Hai selezionato troppe opzioni.

Le risposte corrette sono: It is good to perform transfer learning before fine tuning, as long as there are enough training data.,

In order to perform backpropagation of the maxpooling it is necessary to keep track of the coordinates of all the locations that have propagated the maximum

Domanda 7

Parzialmente corretta

Punteggio ottenuto 0,17 su 2,00

Mark all the sentences that are correct.

Please, refrain from marking answers that are "technically possible in Python", and stick to those answers that a Neural Network expert (as you are expected to be) would give and the solution he would adopt.

Scegli una o più alternative:

- ☐ a. It is possible to train a U-Net architecture via the "fully-convolutionalization process", thus leveraging only annotations on the class the input image belongs to
- ☐ b. It is possible to transform a CNN containing a flattening layer to fully convolutional neural network, as long as the flattening is directly connected to the output layer.
- ☒ c. In order to move to fully convolutional a CNN classifier having a dense layer with N output neurons, it is necessary to include a convolutional layer with L / N filters, where N is the product of spatial dimensions of filters ✗ FALSE: this layer is mapped to a conv layer of L filters
- ☐ d. When designing semantic segmentation networks it is mandatory to include skip connections.
- ☐ e. It is not possible to transform a trained CNN classifier containing a GAP to a Fully convolutional a network
- ☐ f. A trained CNN classifier that is made of conv, maxpool and dense layers can be always modified to return low-resolution heatmaps of class probabilities
- ☐ g. It is possible to transform a trained CNN classifier containing flattening to a Fully convolutional a network
- ☒ h. Segmentation network are not meant to locate objects in the scene ✓ TRUE: as they do not separate connected instances

Your answer is partially correct.

Hai selezionato correttamente 1.

Le risposte corrette sono: A trained CNN classifier that is made of conv, maxpool and dense layers can be always modified to return low-resolution heatmaps of class probabilities, Segmentation network are not meant to locate objects in the scene, It is possible to transform a trained CNN classifier containing flattening to a Fully convolutional a network

Domanda **8**

Completo

Non valutata

Consider the following code

Use the following text box to do calculations and enter your answer as a "backup option". This question won't be assessed except made for "disaster recovery conditions"

```
import tensorflow as tf
tfkl = tf.keras.layers
tfk = tf.keras

def get_model_27(input_shape, output_number):

    # Output Layer
    input_layer = tfkl.Input(shape=input_shape, name='input_layer')

    # Block 1
    c1 = tfkl.Conv2D(64, kernel_size=7, padding='same', activation='relu', name='block_1_conv_1')(input_layer)
    c1 = tfkl.MaxPooling2D(name='block_1_mp')(c1)
    c1 = tfkl.Dropout(0.2, name='block_1_dropout')(c1)
    s1 = tfkl.Conv2D(128, kernel_size=1, padding='same', activation='relu', name='block_1_conv_2')(c1)
    s1 = tfkl.Conv2D(64, kernel_size=7, padding='same', activation='relu', name='block_1_conv_3')(s1)
    s1 = tfkl.Add(name='block_1_add')([c1,s1])

    # Block 2
    c2 = tfkl.Conv2D(128, kernel_size=7, padding='same', activation='relu', name='block_2_conv_1')(s1)
    c2 = tfkl.MaxPooling2D(name='block_2_mp')(c2)
    c2 = tfkl.Dropout(0.2, name='block_2_dropout')(c2)
    s2 = tfkl.Conv2D(256, kernel_size=1, padding='same', activation='relu', name='block_2_conv_2')(c2)
    s2 = tfkl.Conv2D(128, kernel_size=7, padding='same', activation='relu', name='block_2_conv_3')(s2)
    s2 = tfkl.Add(name='block_2_add')([c2,s2])

    s2 = tfkl.GlobalAveragePooling2D(name='gap')(s2)
    s2 = tfkl.Dense(64, name='dense_1')(s2)
    s2 = tfkl.BatchNormalization(name='bn')(s2)
    s2 = tfkl.Dense(64, name='dense_2')(s2)

    # Output Layer
    output_layer = tfkl.Dense(output_number, activation="linear", name='output_layer')(s2)

    model = tf.keras.Model(inputs=input_layer, outputs=output_layer)
    model.compile(
        loss = tfk.losses.MeanSquaredError(),
        optimizer = tfk.optimizers.Adam(),
        metrics = [
            tfk.metrics.MeanAbsoluteError(),
            tfk.metrics.MeanAbsolutePercentageError()
        ]
    )
    return model
```

Consider the following command being executed:

```
input_shape = (224,112,3)
output_number = 6
model = get_model_27(input_shape=input_shape, output_number=output_number)
model.summary()
```

Use the following textbox and the following schema to prepare the model summary requested in the next input

Layer (type)	Output Shape	P. FORMULA	P. NR#	Connected to
=====				
input_layer (InputLayer)	(None, ..., ..., ...)	...	...	NONE
block_1_conv_1 (Conv2D)	(None, ..., ..., ...)	...	...	input_layer
block_1_mp (MaxPooling2D)	(None, ..., ..., ...)	...	...	...
block_1_dropout (Dropout)	(None, ..., ..., ...)	...	...	...
block_1_conv_2 (Conv2D)	(None, ..., ..., ...)	...	...	...
block_1_conv_3 (Conv2D)	(None, ..., ..., ...)	...	...	...
block_1_add (Add)	(None, ..., ..., ...)	...	...	...
block_2_conv_1 (Conv2D)	(None, ..., ..., ...)	...	...	...
block_2_mp (MaxPooling2D)	(None, ..., ..., ...)	...	...	...
block_2_dropout (Dropout)	(None, ..., ..., ...)	...	...	...
block_2_conv_2 (Conv2D)	(None, ..., ..., ...)	...	...	...
block_2_conv_3 (Conv2D)	(None, ..., ..., ...)	...	...	...
block_2_add (Add)	(None, ..., ..., ...)	...	...	...
gap	(None, ..., ..., ...)	...	...	...
dense_1 (Dense)	(None, ..., ..., ...)	...	...	...
bn (BatchNormalization)	(None, ..., ..., ...)	...	...	...
dense_2 (Dense)	(None, ..., ..., ...)	...	...	...
output_layer (Dense)	(None, ..., ..., ...)	...	...	...
=====				
Total params: ...				
Trainable params: ...				
Non-trainable params: ...				

Domanda 9

Risposta corretta

Punteggio ottenuto 4,00 su 4,00

Remember that this question will be automatically rated, therefore you have to consider the following details very carefully:

- when the activations become a vector, these are stacked along the third dimension (the channel, C). Therefore, if your latent representation becomes a 3 element vector, enter as sizes H:1 W:1 C:3.
- Enter digits **only** and do not use "." or "," or "-" or "[". Say write "1234" and not "1,234" or "1.234". Write "0", not "-" or "[".
- Do not enter formulas, just resulting numbers
- Do not enter empty spaces... please double check
- you don't need to fill in connections, but please consider them.
- Wrong answers can result in negative points

All the numbers have to be entered here, the text in the previous question is just meant for you to design the solution.

Layer (type)	Output Shape	Param NR
=====		
input_layer (InputLayer)	H: 224 ✓ , W: 112 ✓ , C: 3 ✓	Params #: 0 ✓
block_1_conv_1 (Conv2D)	H: 224 ✓ , W: 112 ✓ , C: 64 ✓	Params #: 9472 ✓
block_1_mp (MaxPooling2D)	H: 112 ✓ , W: 56 ✓ , C: 64 ✓	Params #: 0 ✓
block_1_dropout (Dropout)	H: 112 ✓ , W: 56 ✓ , C: 64 ✓	Params #: 0 ✓
block_1_conv_2 (Conv2D)	H: 112 ✓ , W: 56 ✓ , C: 128 ✓	Params #: 8320 ✓
block_1_conv_3 (Conv2D)	H: 112 ✓ , W: 56 ✓ , C: 64 ✓	Params #: 401472 ✓
block_1_add (Add)	H: 112 ✓ , W: 56 ✓ , C: 64 ✓	Params #: 0 ✓
block_2_conv_1 (Conv2D)	H: 112 ✓ , W: 56 ✓ , C: 128 ✓	Params #: 401536 ✓
block_2_mp (MaxPooling2D)	H: 56 ✓ , W: 28 ✓ , C: 128 ✓	Params #: 0 ✓
block_2_dropout (Dropout)	H: 56 ✓ , W: 28 ✓ , C: 128 ✓	Params #: 0 ✓
block_2_conv_2 (Conv2D)	H: 56 ✓ , W: 28 ✓ , C: 256 ✓	Params #: 33024 ✓
block_2_conv_3 (Conv2D)	H: 56 ✓ , W: 28 ✓ , C: 128 ✓	Params #: 1605760 ✓
block_2_add (Add)	H: 56 ✓ , W: 28 ✓ , C: 128 ✓	Params #: 0 ✓
gap	H: 1 ✓ , W: 1 ✓ , C: 128 ✓	Params #: 0 ✓
dense_1 (Dense)	H: 1 ✓ , W: 1 ✓ , C: 64 ✓	Params #: 8256 ✓
bn (BatchNormalization)	H: 1 ✓ , W: 1 ✓ , C: 64 ✓	Params #: 256 ✓
dense_2 (Dense)	H: 1 ✓ , W: 1 ✓ , C: 64 ✓	Params #: 4160 ✓
output_layer (Dense)	H: 1 ✓ , W: 1 ✓ , C: 6 ✓	Params #: 390 ✓
=====		
Total params:	2472646 ✓	
Trainable params:	2472518 ✓	
Non-trainable params:	128 ✓	
=====		

Layer (type)	Output Shape	Param #	Connected to
=====			
input_layer (InputLayer)	[(None, 224, 112, 3)]	0	□

block_1_conv_1 (Conv2D)	(None, 224, 112, 64)	9472	['input_layer[0][0]']
block_1_mp (MaxPooling2D)	(None, 112, 56, 64)	0	['block_1_conv_1[0][0]']
block_1_dropout (Dropout)	(None, 112, 56, 64)	0	['block_1_mp[0][0]']
block_1_conv_2 (Conv2D)	(None, 112, 56, 128)	8320	['block_1_dropout[0][0]']
block_1_conv_3 (Conv2D)	(None, 112, 56, 64)	401472	['block_1_conv_2[0][0]']
block_1_add (Add)	(None, 112, 56, 64)	0	['block_1_dropout[0][0]','block_1_conv_3[0][0]']
block_2_conv_1 (Conv2D)	(None, 112, 56, 128)	401536	['block_1_add[0][0]']
block_2_mp (MaxPooling2D)	(None, 56, 28, 128)	0	['block_2_conv_1[0][0]']
block_2_dropout (Dropout)	(None, 56, 28, 128)	0	['block_2_mp[0][0]']
block_2_conv_2 (Conv2D)	(None, 56, 28, 256)	33024	['block_2_dropout[0][0]']
block_2_conv_3 (Conv2D)	(None, 56, 28, 128)	1605760	['block_2_conv_2[0][0]']
block_2_add (Add)	(None, 56, 28, 128)	0	['block_2_dropout[0][0]','block_2_conv_3[0][0]']
gap (GlobalAveragePooling2)	(None, 128)	0	['block_2_add[0][0]']
dense_1 (Dense)	(None, 64)	8256	['gap[0][0]']
bn (BatchNormalization)	(None, 64)	256	['dense_1[0][0]']
dense_2 (Dense)	(None, 64)	4160	['bn[0][0]']
output_layer (Dense)	(None, 6)	390	['dense_2[0][0]']

=====

Total params: 2472646 (9.43 MB)

Trainable params: 2472518 (9.43 MB)

Non-trainable params: 128 (512.00 Byte)

Domanda 10

Parzialmente corretta

Punteggio ottenuto 1,50 su 2,00

Mark all the tasks that the above network can be trained for.

Please, refrain from marking answers that are "technically and eventually possible in Python", and stick to those tasks that a Neural Network expert (as you are expected to be) would use this network for.

Needless to say, assume you are always provided with the required training data for these tasks, while you cannot change the architecture and training options for the above model.

Scegli una o più alternative:

- ☐ a. Given a picture of a car, estimate its weight, its speed, and localize it through a bounding box.
- ☐ b. This is an instance segmentation network for cells in a biopsy image. However, it can count up to 6 cells.
- ☐ c. Given a picture taken from a driving car, this network classifies all pixels in categories like: car, person, bicycle, truck, traffic light and background
- ☒ d. Counting the number of cats, of dogs, of traffic lights, of bicycles, of cars and of persons in a picture taken from "Street View" ✓ TRUE: this is a 6-target regression problem
- ☐ e. Given a picture of a human face, drawing a bounding box around the face and say whether the person holds at least one degree or not
- ☒ f. Given a picture of a six-person family, this network estimates the age of the six components. ✓ TRUE: this is a 6-target regression problem
- ☐ g. This is a network performing weather forecasting on the Lombardia region, squeezed in an image 224x112.
- ☐ h. Detecting persons and cats, and for each of these return a bounding box.
- ☐ i. Classify the the digits from 0 to 5 in from a subset of MNIST dataset.
- ☒ j. Given the picture of a human face, estimate the position of the tip of ears and the tip of nose. ✓ TRUE: this is regression over 6 coordinates in the space
- ☐ k. Given a picture of a six-person family, this network returns 6 text fields corresponding to name of the six components.
- ☐ l. Given a picture of a self-driving car, this network estimates the height, width, and length of the car, together with its price in dollars.

Your answer is partially correct.

Hai selezionato correttamente 3.

Le risposte corrette sono:

Counting the number of cats, of dogs, of traffic lights, of bicycles, of cars and of persons in a picture taken from "Street View" ,

Given a picture of a car, estimate its weight, its speed, and localize it through a bounding box., Given the picture of a human face, estimate the position of the tip of ears and the tip of nose.,

Given a picture of a six-person family, this network estimates the age of the six components.